Solution: Longest Common Subsequence

Problem Statement

Given two strings 's1' and 's2', find the length of the longest subsequence which is common in both the strings.

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

```
Input: s1 = "abdca"
       s2 = "cbda"
Output: 3
Explanation: The longest common subsequence is "bda".
```

Example 2:

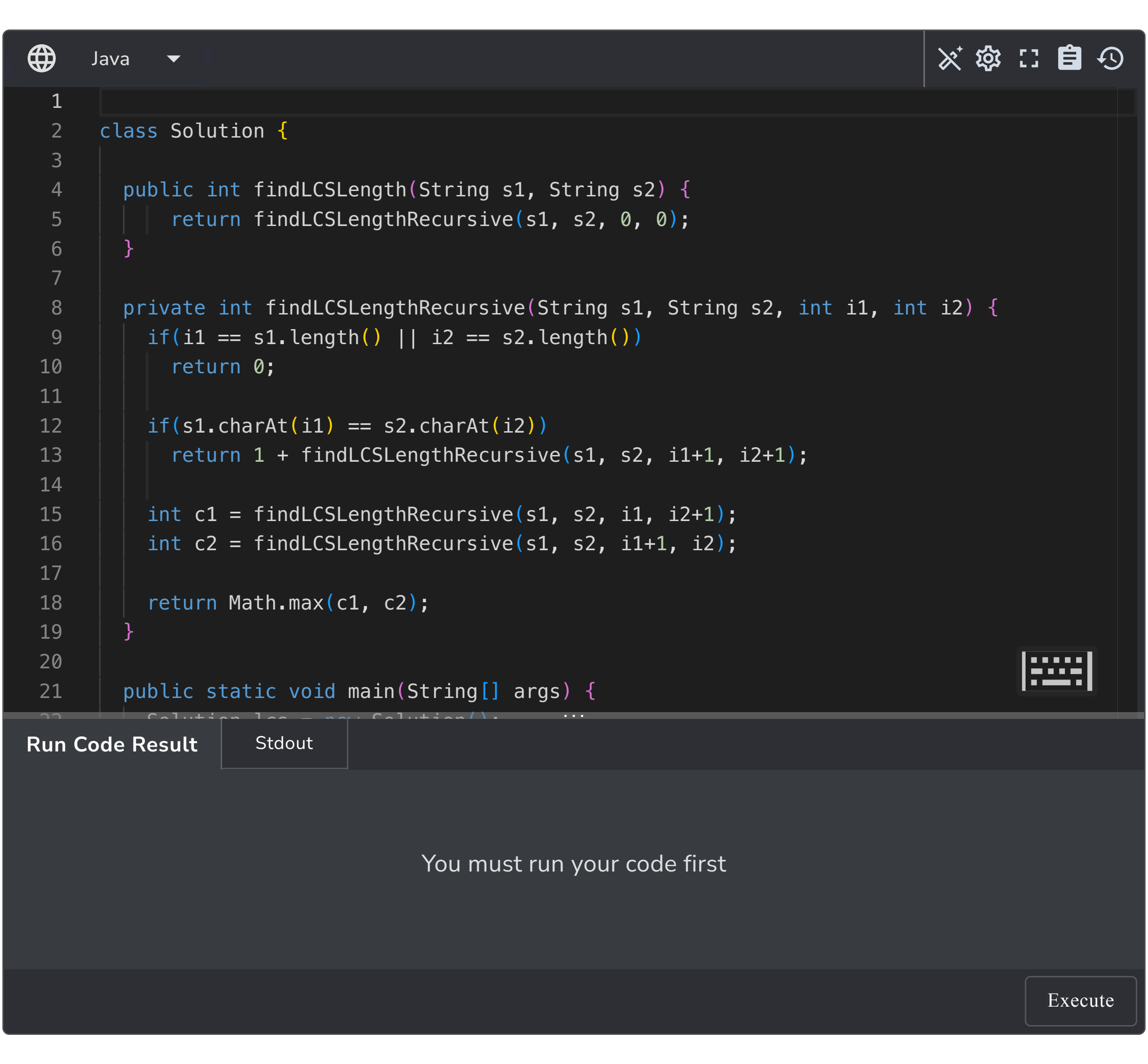
```
Input: s1 = "passport"
       s2 = "ppsspt"
Output: 5
Explanation: The longest common subsequence is "psspt".
```

Basic Solution

A basic brute-force solution could be to try all subsequences of 's1' and 's2' to find the longest one. We can match both the strings one character at a time. So for every index 'i' in 's1' and 'j' in 's2' we must choose between:

1. If the character `s1[i]` matches `s2[j]`, we can recursively match for the remaining lengths.
2. If the character `s1[i]` does not match `s2[j]`, we will start two new recursive calls by skipping one character separately from each string.

Here is the code:



```java
class Solution {

  public int findLCSLength(String s1, String s2) {
    return findLCSLengthRecursive(s1, s2, 0, 0);
  }

  private int findLCSLengthRecursive(String s1, String s2, int i1, int i2) {
    if(i1 == s1.length() || i2 == s2.length())
      return 0;

    if(s1.charAt(i1) == s2.charAt(i2))
      return 1 + findLCSLengthRecursive(s1, s2, i1+1, i2+1);

    int c1 = findLCSLengthRecursive(s1, s2, i1, i2+1);
    int c2 = findLCSLengthRecursive(s1, s2, i1+1, i2);

    return Math.max(c1, c2);
  }

  public static void main(String[] args) {
```

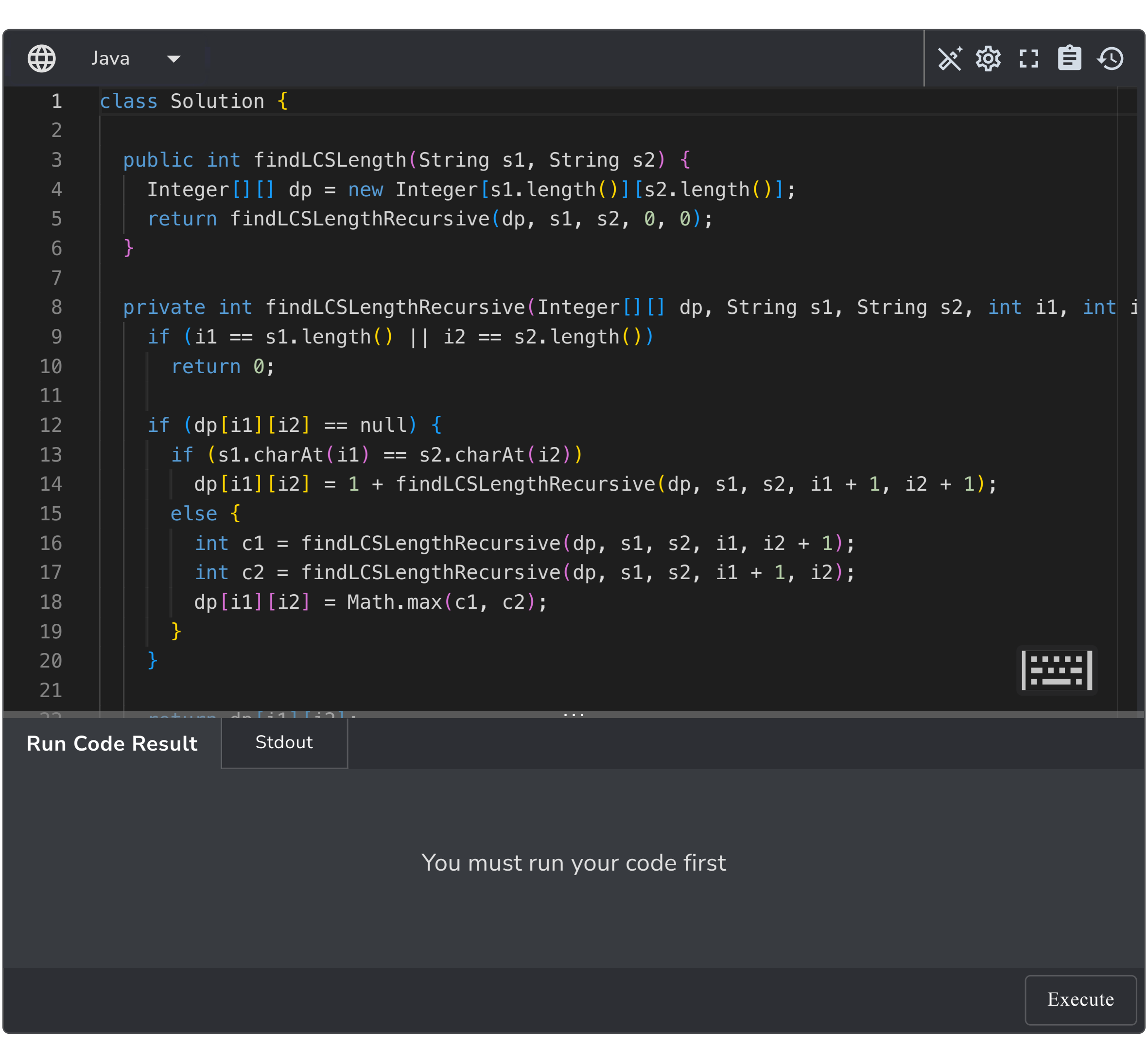
The time complexity of the above algorithm is exponential $O(2^{m+n})$, where 'm' and 'n' are the lengths of the two input strings. The space complexity is $O(n+m)$ which is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems.

The two changing values to our recursive function are the two indexes, i1 and i2. Therefore, we can store the results of all the subproblems in a two-dimensional array. (Another alternative could be to use a hash-table whose key would be a string (i1 + "|" + i2)).

Here is the code:



```java
class Solution {

  public int findLCSLength(String s1, String s2) {
    Integer[][] dp = new Integer[s1.length()][s2.length()];
    return findLCSLengthRecursive(dp, s1, s2, 0, 0);
  }

  private int findLCSLengthRecursive(Integer[][] dp, String s1, String s2, int i1, int i
    if (i1 == s1.length() || i2 == s2.length())
      return 0;

    if (dp[i1][i2] == null) {
      if (s1.charAt(i1) == s2.charAt(i2))
        dp[i1][i2] = 1 + findLCSLengthRecursive(dp, s1, s2, i1 + 1, i2 + 1);
      else {
        int c1 = findLCSLengthRecursive(dp, s1, s2, i1, i2 + 1);
        int c2 = findLCSLengthRecursive(dp, s1, s2, i1 + 1, i2);
        dp[i1][i2] = Math.max(c1, c2);
      }
    }

```

Bottom-up Dynamic Programming

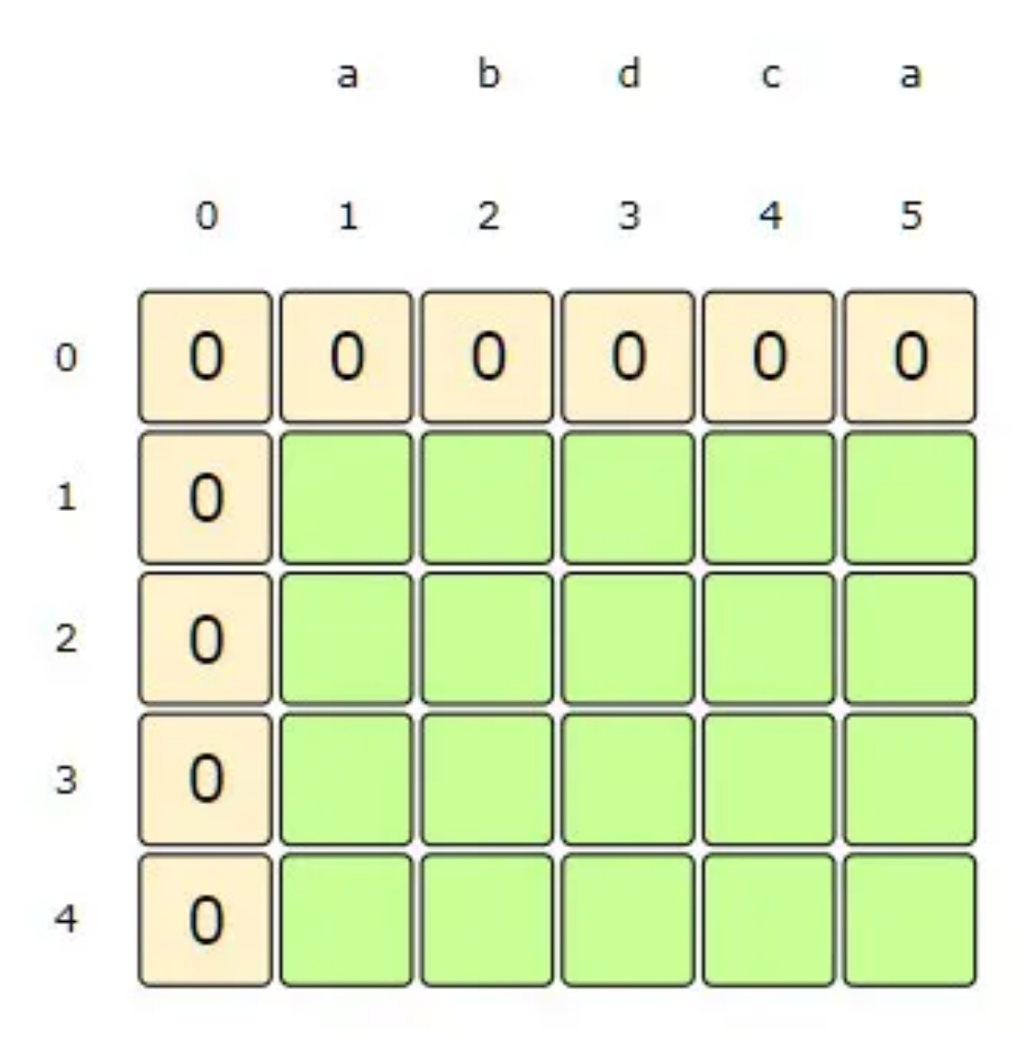
Since we want to match all the subsequences of the given two strings, we can use a two-dimensional array to store our results. The lengths of the two strings will define the size of the array's two dimensions. So for every index 'i' in string 's1' and 'j' in string 's2', we will choose one of the following two options:

1. If the character `s1[i]` matches `s2[j]`, the length of the common subsequence would be one plus the length of the common subsequence till the `i-1` and `j-1` indexes in the two respective strings.
2. If the character `s1[i]` does not match `s2[j]`, we will take the longest subsequence by either skipping ith or jth character from the respective strings.

So our recursive formula would be:

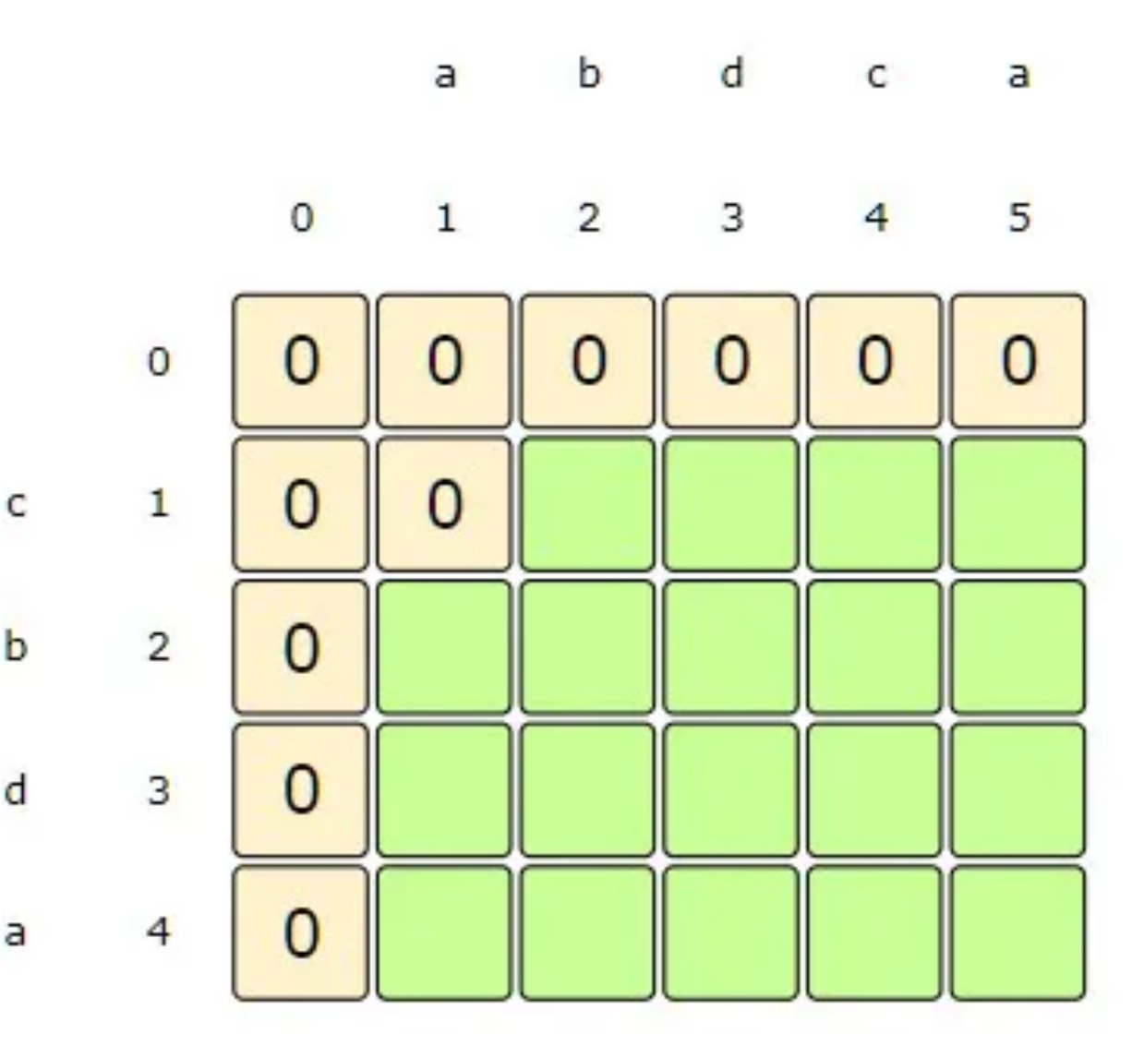
```
if s1[i] == s2[j]
  dp[i][j] = 1 + dp[i-1][j-1]
else
  dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Let's draw this visually for "abdca" and "cbda". Starting with a subsequence of zero lengths, if any string has zero length then the common subsequence will be of zero length:



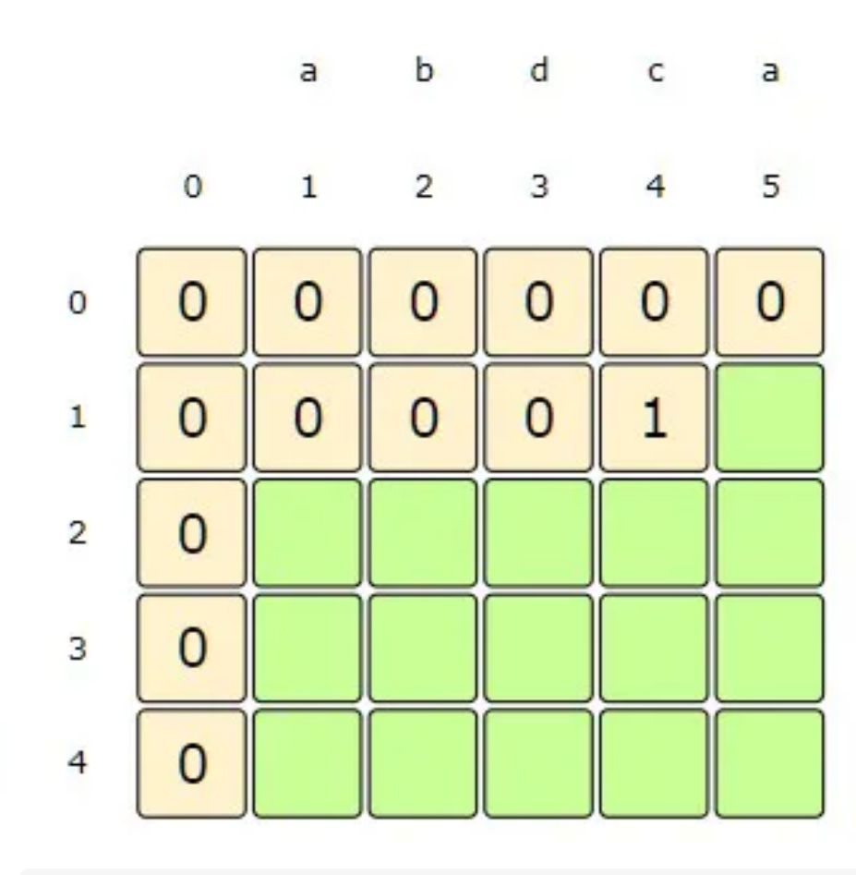
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0					
b	2	0					
d	3	0					
a	4	0					

i:0, j:0-5 and i:0-4, j:0 => dp[i][j] = 0, as we don't have any common subsequence when one of the string is of zero length



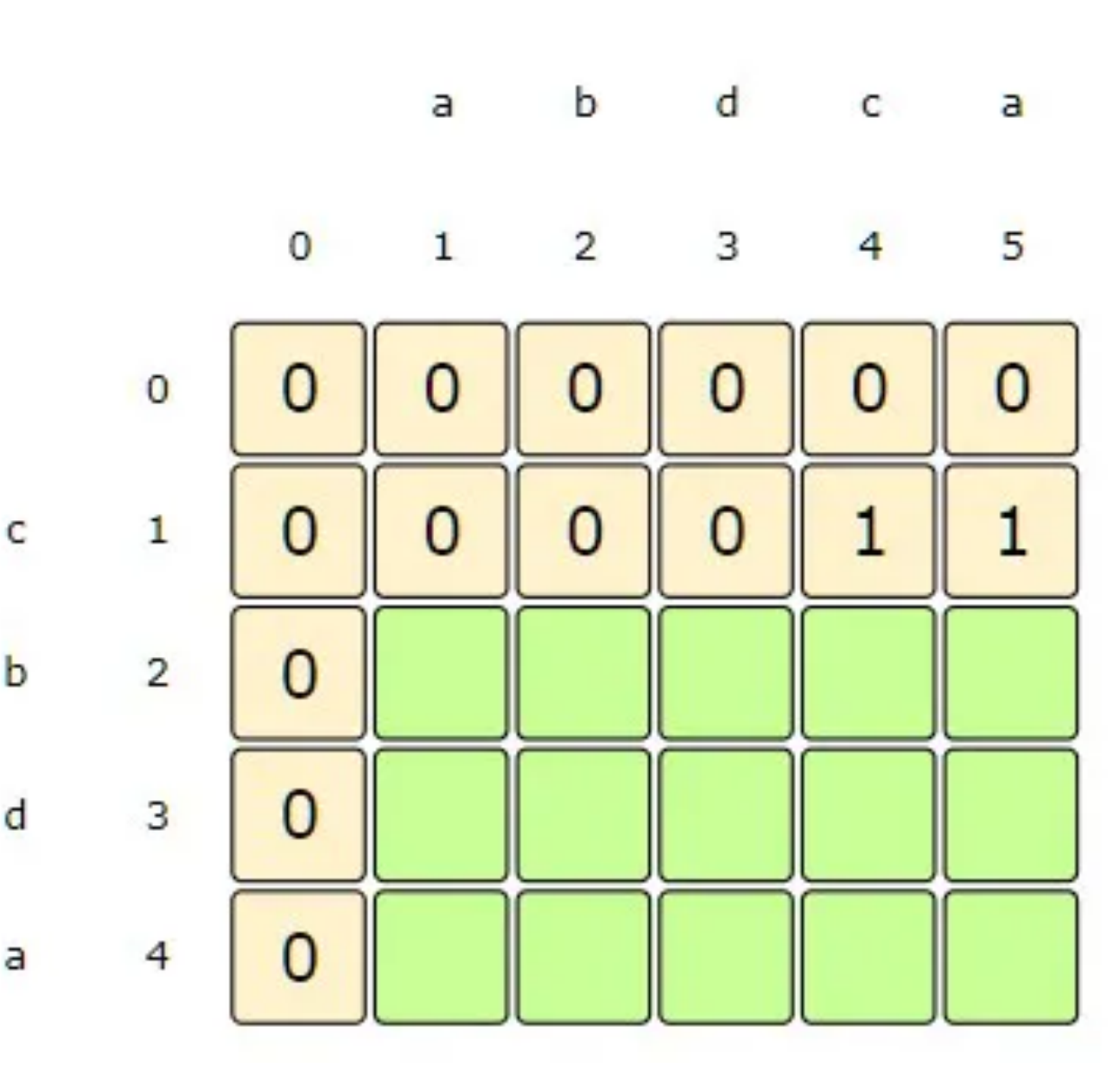
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0				
b	2	0					
d	3	0					
a	4	0					

i:1, j:1 => dp[i][j] = max(dp[i-1][j], dp[i][j-1]), as s1[i] != s2[j]



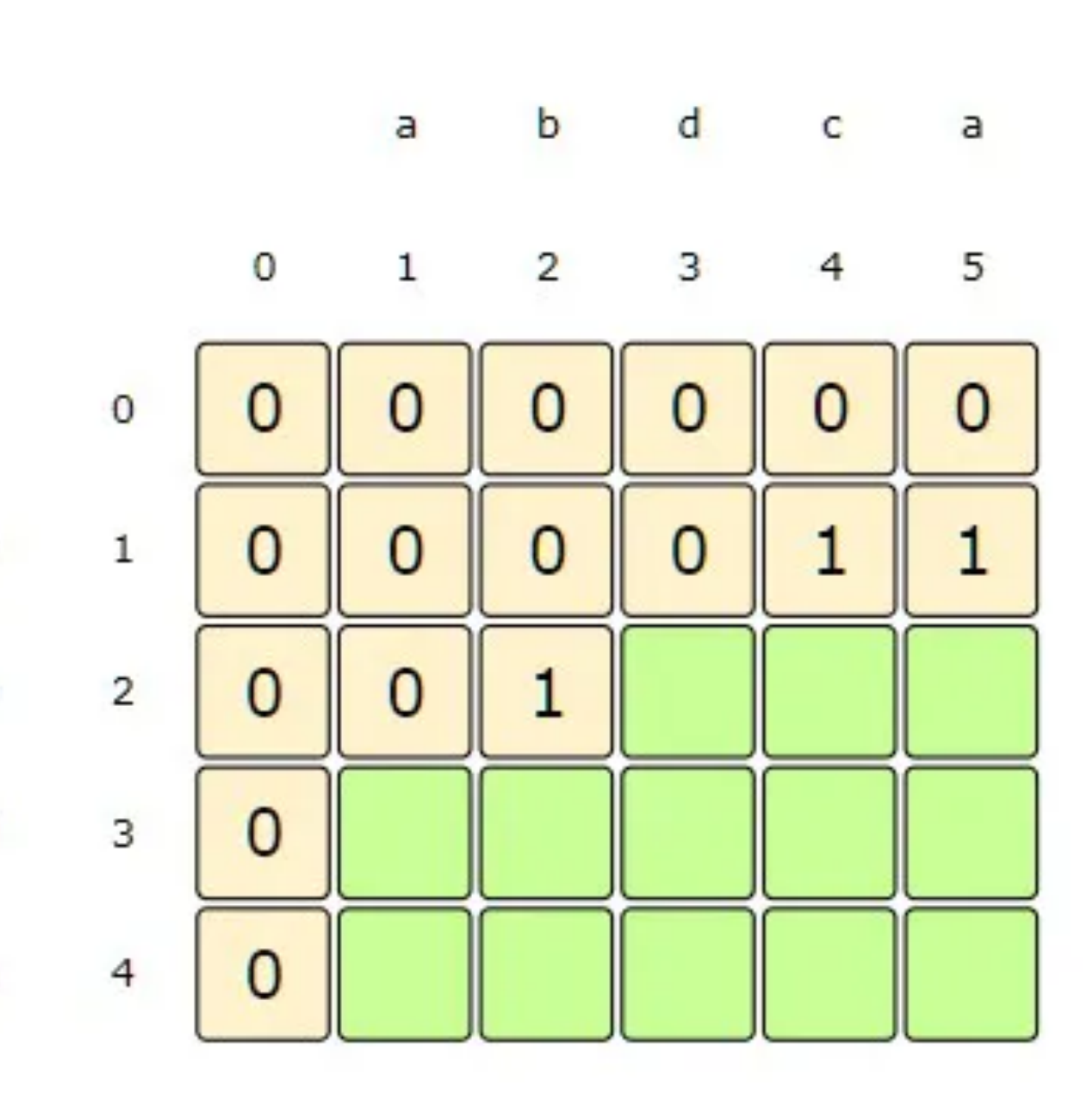
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	
b	2	0					
d	3	0					
a	4	0					

i:1, j:4 => dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]



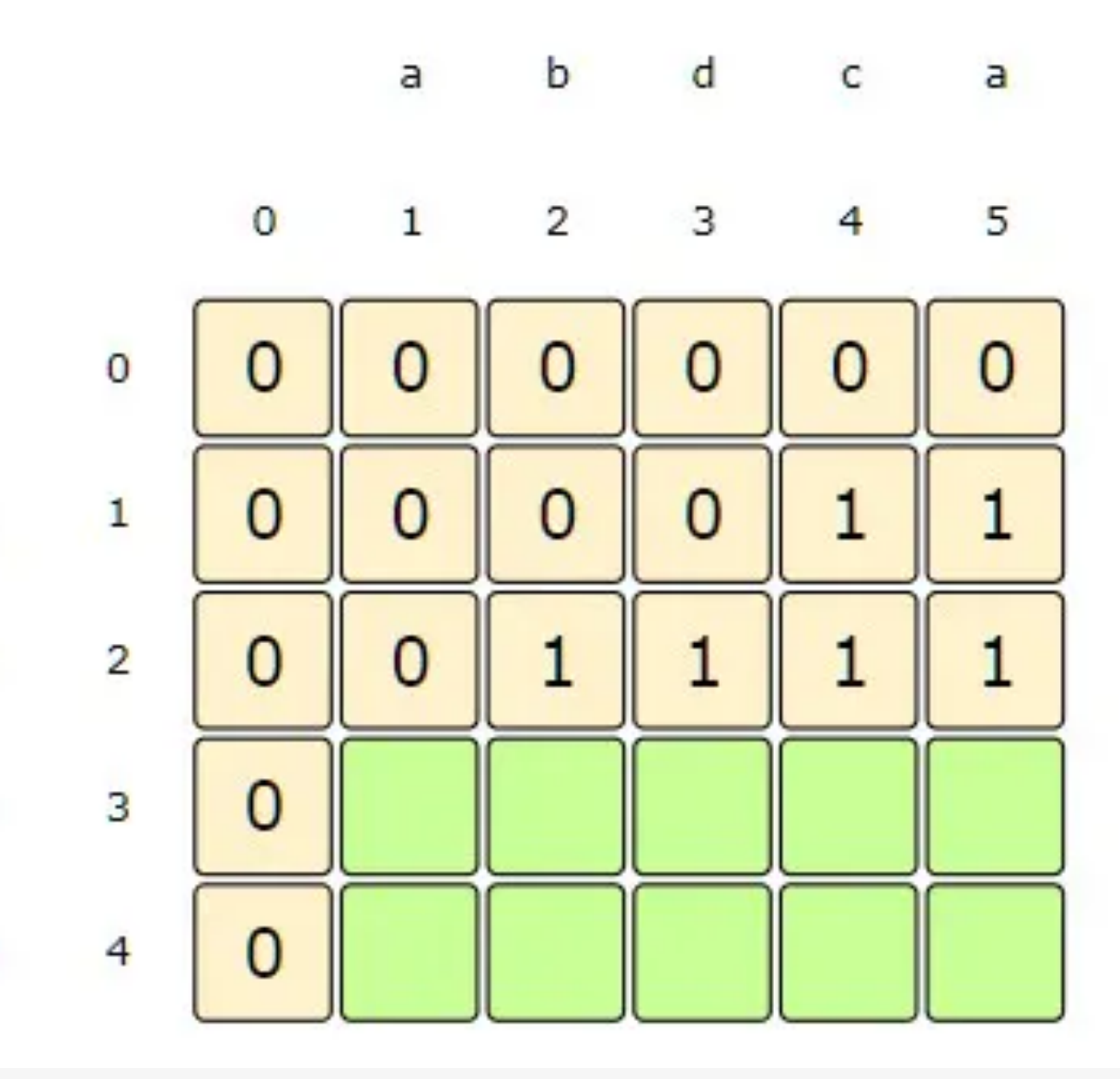
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	1
b	2	0					
d	3	0					
a	4	0					

i:1, j:5 => dp[i][j] = max(dp[i-1][j], dp[i][j-1]), as s1[i] != s2[j]



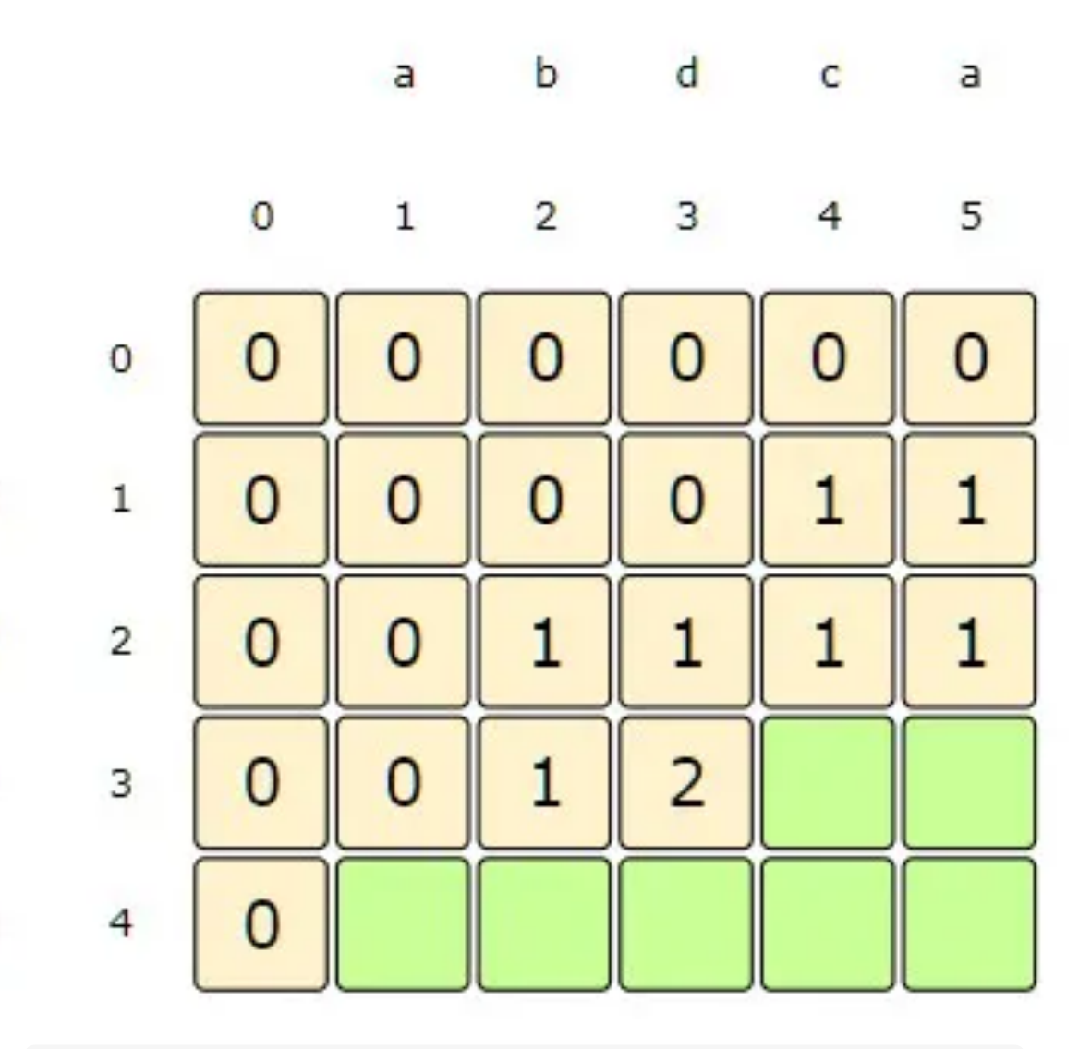
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	1
b	2	0	0	1			
d	3	0					
a	4	0					

i:2, j:2=> dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]



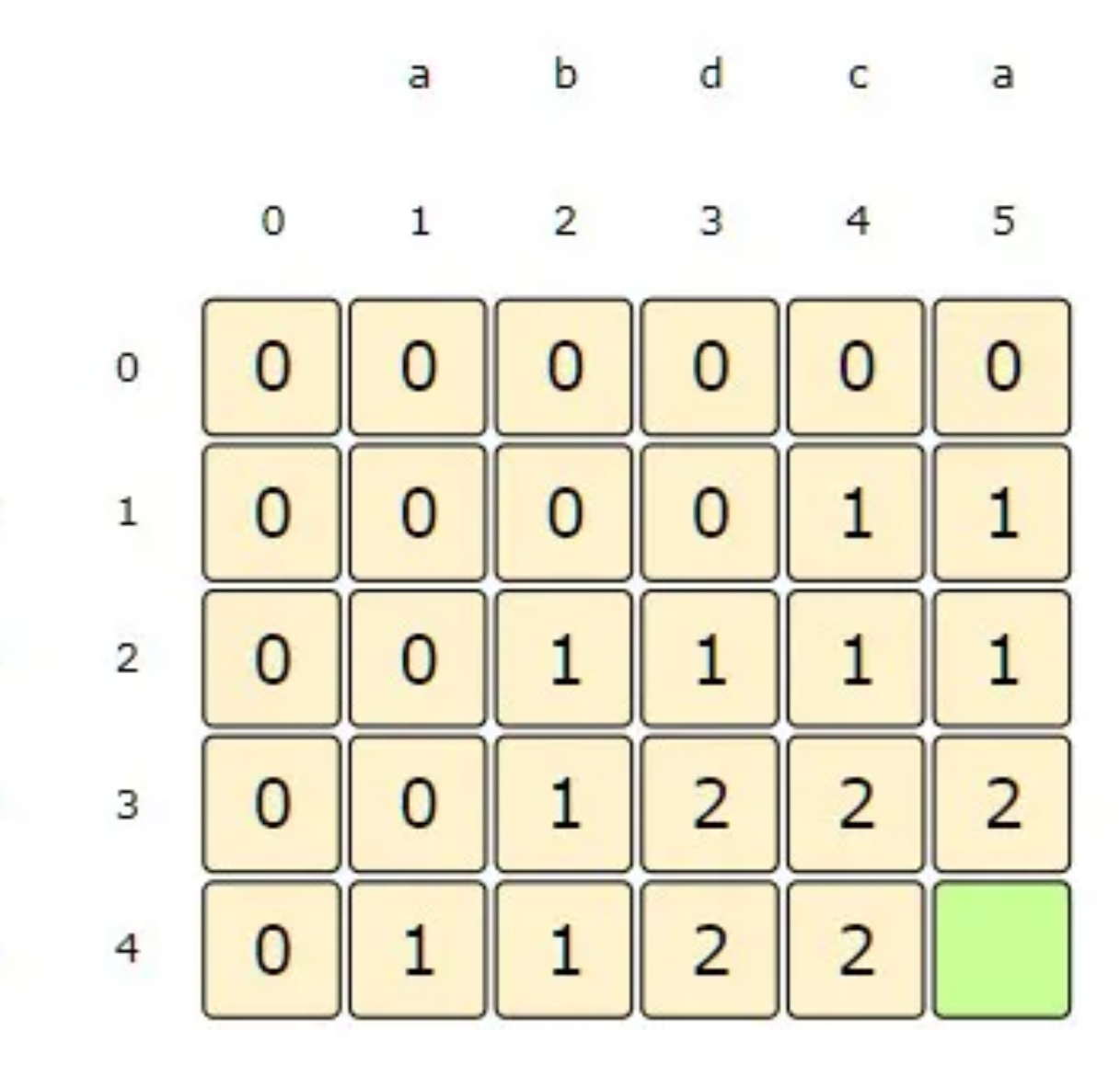
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	1
b	2	0	0	1	1	1	1
d	3	0					
a	4	0					

i:2, j:3-5 => dp[i][j] = max(dp[i-1][j], dp[i][j-1]), as s1[i] != s2[j]



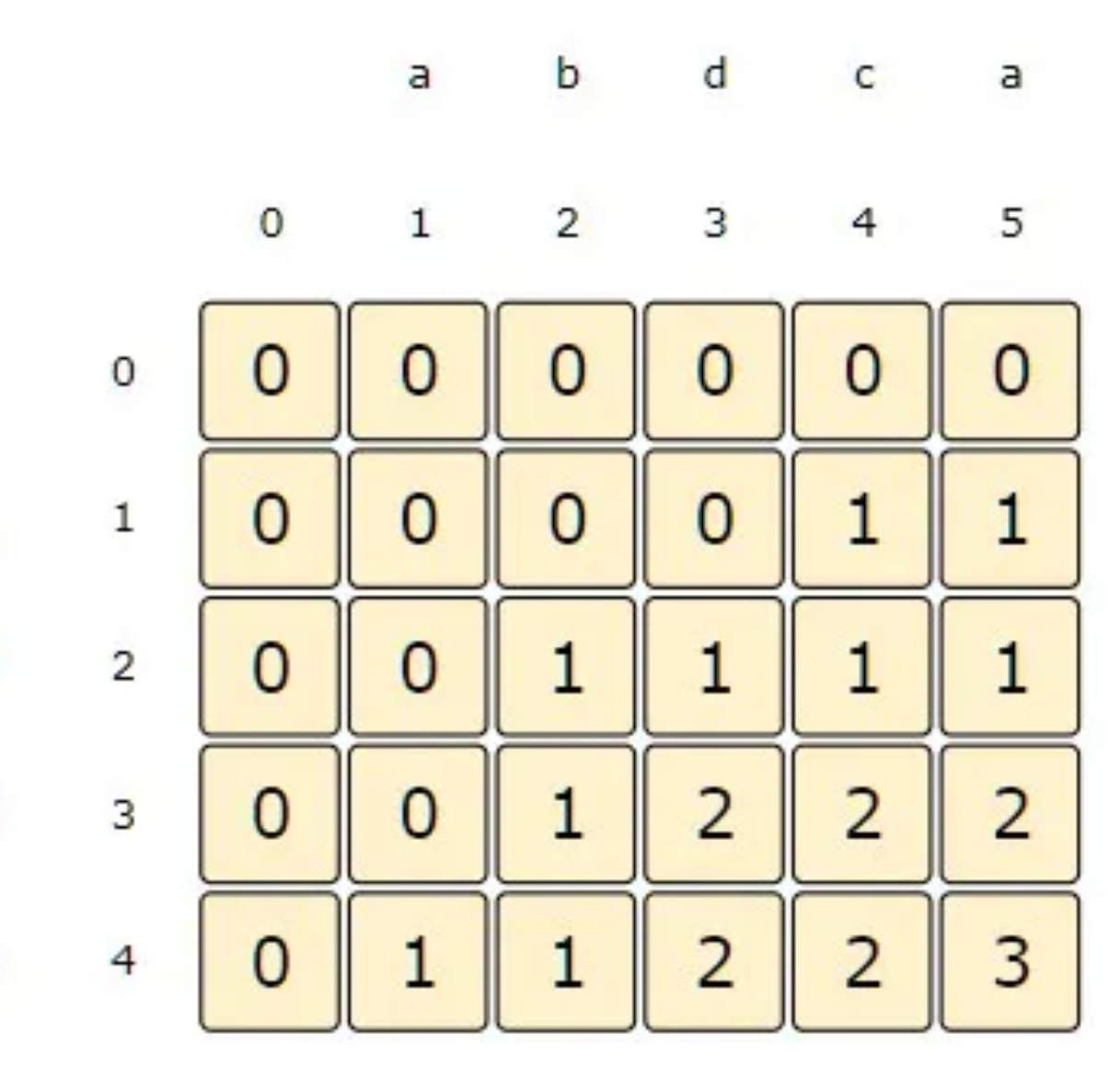
			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	1
b	2	0	0	1	1	1	1
d	3	0	0	1	2		
a	4	0					

i:3, j:3=> dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]



			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	1
b	2	0	0	1	1	1	1
d	3	0	0	1	2	2	2
a	4	0	1	1	2	2	

i:4, j:5=> dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]

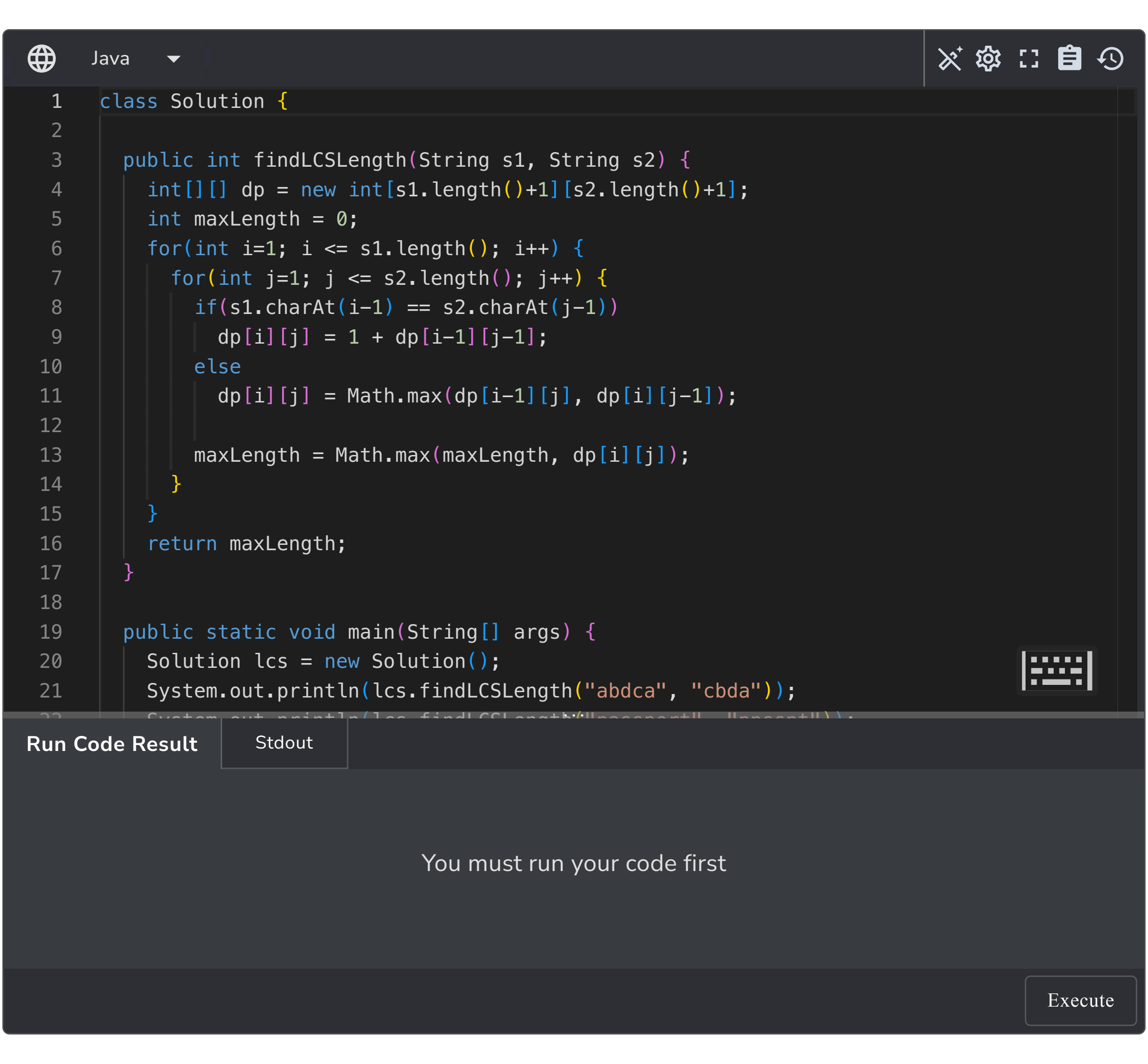


			a	b	d	c	a
		0	1	2	3	4	5
	0	0	0	0	0	0	0
c	1	0	0	0	0	1	1
b	2	0	0	1	1	1	1
d	3	0	0	1	2	2	2
a	4	0	1	1	2	2	3

i:4, j:5=> dp[i][j] = 1 + dp[i-1][j-1], as s1[i] == s2[j]

From the above visualization, we can clearly see that the longest common subsequence is of length '3' -- as shown by `dp[4][5]`.

Here is the code for our bottom-up dynamic programming approach:



```java
class Solution {

  public int findLCSLength(String s1, String s2) {
    int[][] dp = new int[s1.length()+1][s2.length()+1];
    int maxLength = 0;
    for(int i=1; i <= s1.length(); i++) {
      for(int j=1; j <= s2.length(); j++) {
        if(s1.charAt(i-1) == s2.charAt(j-1))
          dp[i][j] = 1 + dp[i-1][j-1];
        else
          dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);

        maxLength = Math.max(maxLength, dp[i][j]);
      }
    }
    return maxLength;
  }

  public static void main(String[] args) {
    Solution lcs = new Solution();
    System.out.println(lcs.findLCSLength("abdca", "cbda"));
```

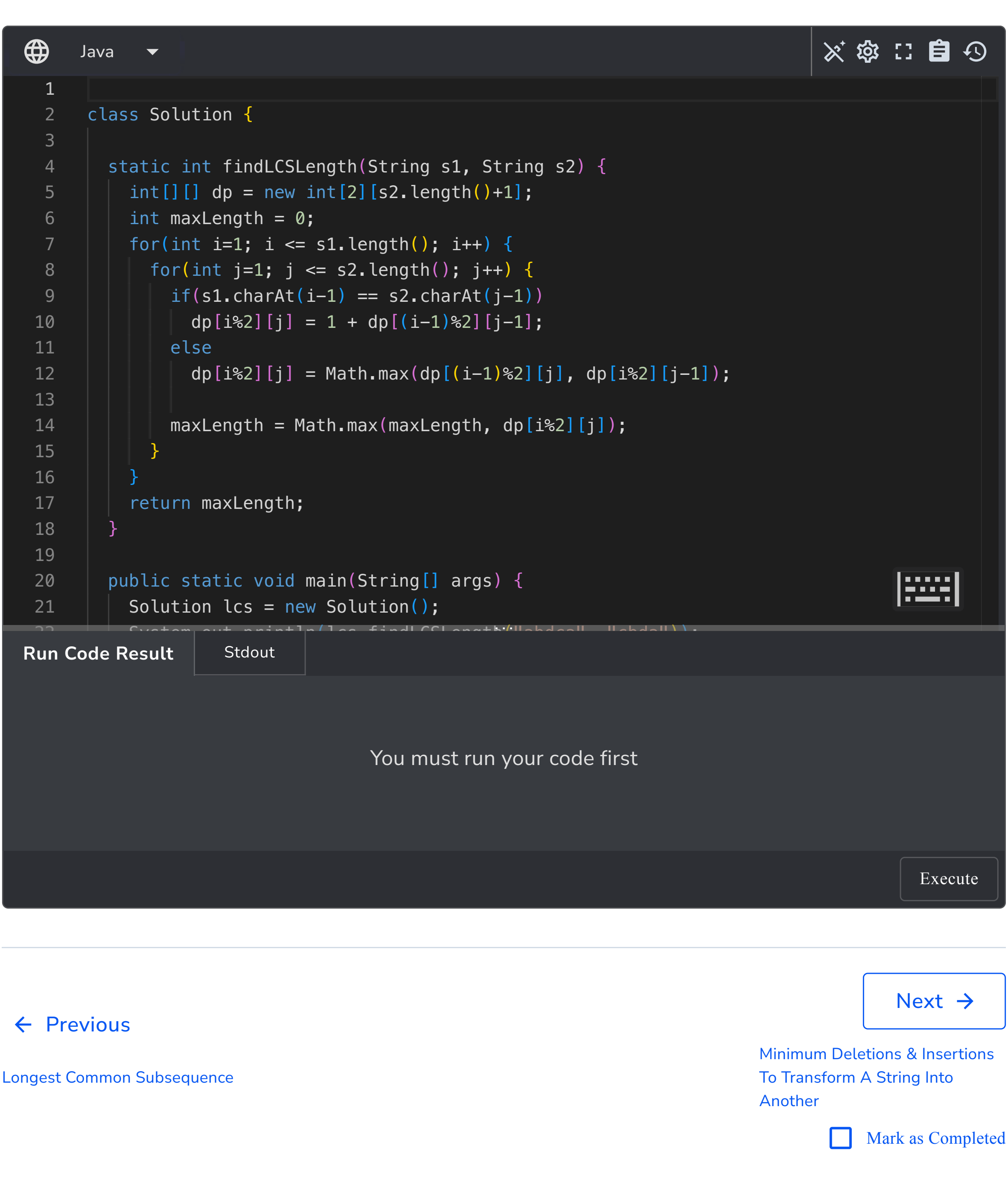
The time and space complexity of the above algorithm is $O(m*n)$, where 'm' and 'n' are the lengths of the two input strings.

Challenge

Can we further improve our bottom-up DP solution? Can you find an algorithm that has $O(n)$ space complexity?

Hint

We only need one previous row to find the optimal solution!



```java

class Solution {

  static int findLCSLength(String s1, String s2) {
    int[][] dp = new int[2][s2.length()+1];
    int maxLength = 0;
    for(int i=1; i <= s1.length(); i++) {
      for(int j=1; j <= s2.length(); j++) {
        if(s1.charAt(i-1) == s2.charAt(j-1))
          dp[i%2][j] = 1 + dp[(i-1)%2][j-1];
        else
          dp[i%2][j] = Math.max(dp[(i-1)%2][j], dp[i%2][j-1]);

        maxLength = Math.max(maxLength, dp[i%2][j]);
      }
    }
    return maxLength;
  }

  public static void main(String[] args) {
    Solution lcs = new Solution();
```

← Previous
Longest Common Subsequence

Next →
Minimum Deletions & Insertions To Transform A String Into Another

☐ Mark as Completed